

GLOBALRAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY 3
CLIENT 3
INSTRUCTIONS 3
DEVELOPER 4
1. ALGORITHM CIPHER 4
2. CERTIFICATE GENERATION 4
3. DEPLOY CIPHER 4
4. SECURE COMMUNICATIONS 4
5. SECONDARY TESTING 4
6. FUNCTIONAL TESTING 4
7. SUMMARY 4
8. INDUSTRY STANDARD BEST PRACTICES 4

Document Revision History

Version	Date	Author	Comments
1.0	10/17/2026	Keenan Vargas	

Client



Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Keenan Vargas

1. Algorithm Cipher

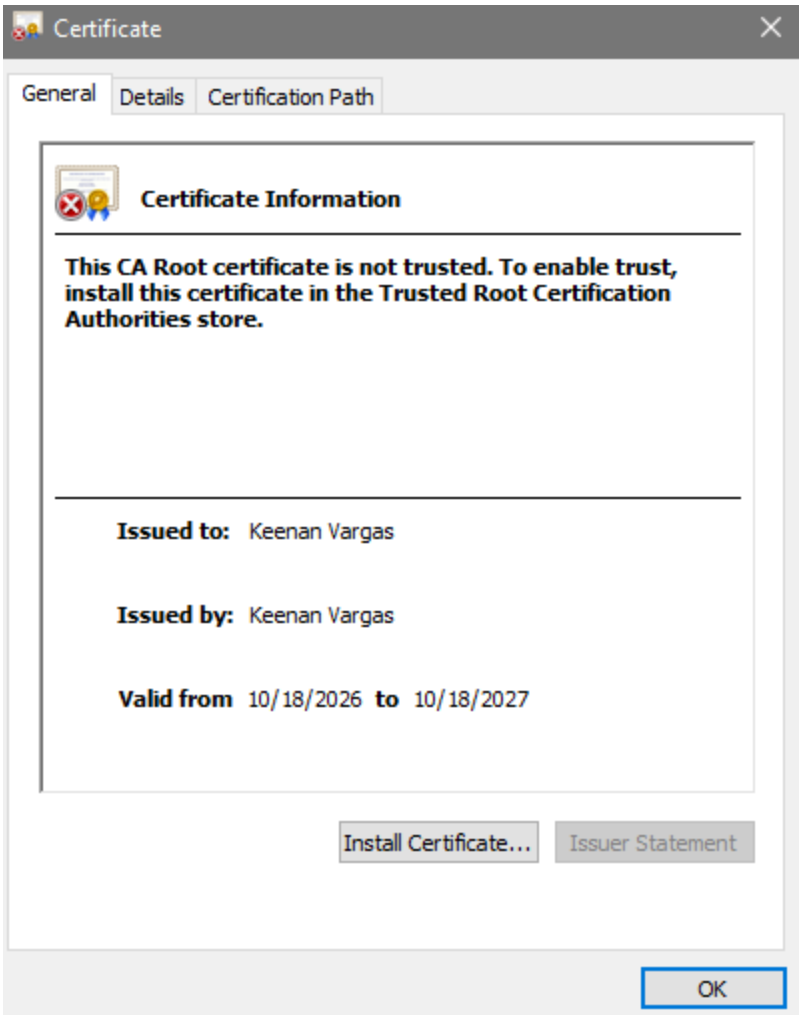
Based on previous assignments, I recommend that the AES cipher is the best cipher algorithm to use and SHA is a great cryptographic hash function. More specifically, the 256-bit AES cipher and the SHA-256 hash function. Global Rain is working with Artemis Financial, and Artemis Financial is a consulting company that develops financial plans for people. Artemis Financial has to deal with sensitive information all the time; this is where AES-256 comes into play. This cipher uses extended key lengths that protect against brute-force attacks. AES-256 is also fully supported in Java standard libraries that Artemis Financial's program uses. This cipher can be used as a data-at-rest encryption, that locks in the data of Artemis Financial and covers the data to an unreadable format, ensuring external entities and factors can only see scrambled characters.

Years ago, encryption was very primitive since many used 56-bit cipher algorithms. However, as time went on, there was a need to improve the length of these cipher algorithms to keep up with the ever-growing internet. Nowadays, we have advanced cipher algorithms that are very length, like the 256-bit AES cipher. One source, Iron-Clad stated, "Most companies that need a massive scale (big banks, and so on) already have a good key management infrastructure in place. The strength of an HMAC is governed by the strength of the underlying hash functions, the size of the key, and the ability to keep the key secret" (Manico et al., 2014). HMAC is a cryptography technique that combines both the hash function and API keys to ensure security and authenticity. So, hash functions are useful for companies like Artemis Financial because they maintain the integrity of the data so that any data change is instantly detected.

Random numbers are used to create unpredictability to prevent any patterns. Symmetric encryptions (like AES) use a single shared secret key that encrypts and decrypts the data, making it very efficient. On the contrary, asymmetric encryptions encrypt with a public key, and decrypt with a private one, making it slower for large amounts of data but ensuring security for transmitting data over the network. Symmetric encryptions are the better alternative since Artemis Financial will receive thousands of requests from users every second.

2. Certificate Generation

Insert a screenshot below of the CER file.




```
(c) Microsoft Corporation. All rights reserved.

C:\>cd "C:\Users\Family Jeudys\Computer Science Related Work\eclipse-workspace\ssl-server_student"

C:\Users\Family Jeudys\Computer Science Related Work\eclipse-workspace\ssl-server_student>"C:\Program Files\Java\jdk-25\bin\keytool.exe" -genkeypair -alias sslserver -keyalg RSA -keysize 2048 -storetype PKCS12 -keystore src\main\resource
s\sslserver.p12 -validity 365
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
  [Unknown]:  Keenan Vargas
What is the name of your organizational unit?
  [Unknown]:  CS
What is the name of your organization?
  [Unknown]:  SNHU
What is the name of your City or Locality?
  [Unknown]:  Boston
What is the name of your State or Province?
  [Unknown]:  MA
What is the two-letter country code for this unit?
  [Unknown]:  US
Is CN=Keenan Vargas, OU=CS, O=SNHU, L=Boston, ST=MA, C=US correct?
  [no]:  yes

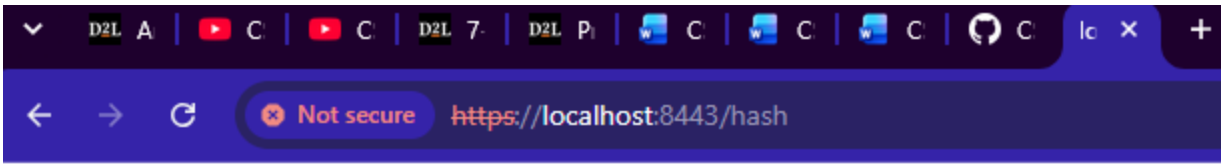
Generating 2,048-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
        for: CN=Keenan Vargas, OU=CS, O=SNHU, L=Boston, ST=MA, C=US

C:\Users\Family Jeudys\Computer Science Related Work\eclipse-workspace\ssl-server_student>"C:\Program Files\Java\jdk-25\bin\keytool.exe" -exportcert -alias sslserver -file src\main\resources\sslserver.cer -keystore src\main\resources\ssl
server.p12
Enter keystore password:
Certificate stored in file <src\main\resources\sslserver.cer>

C:\Users\Family Jeudys\Computer Science Related Work\eclipse-workspace\ssl-server_student>
```

3. Deploy Cipher

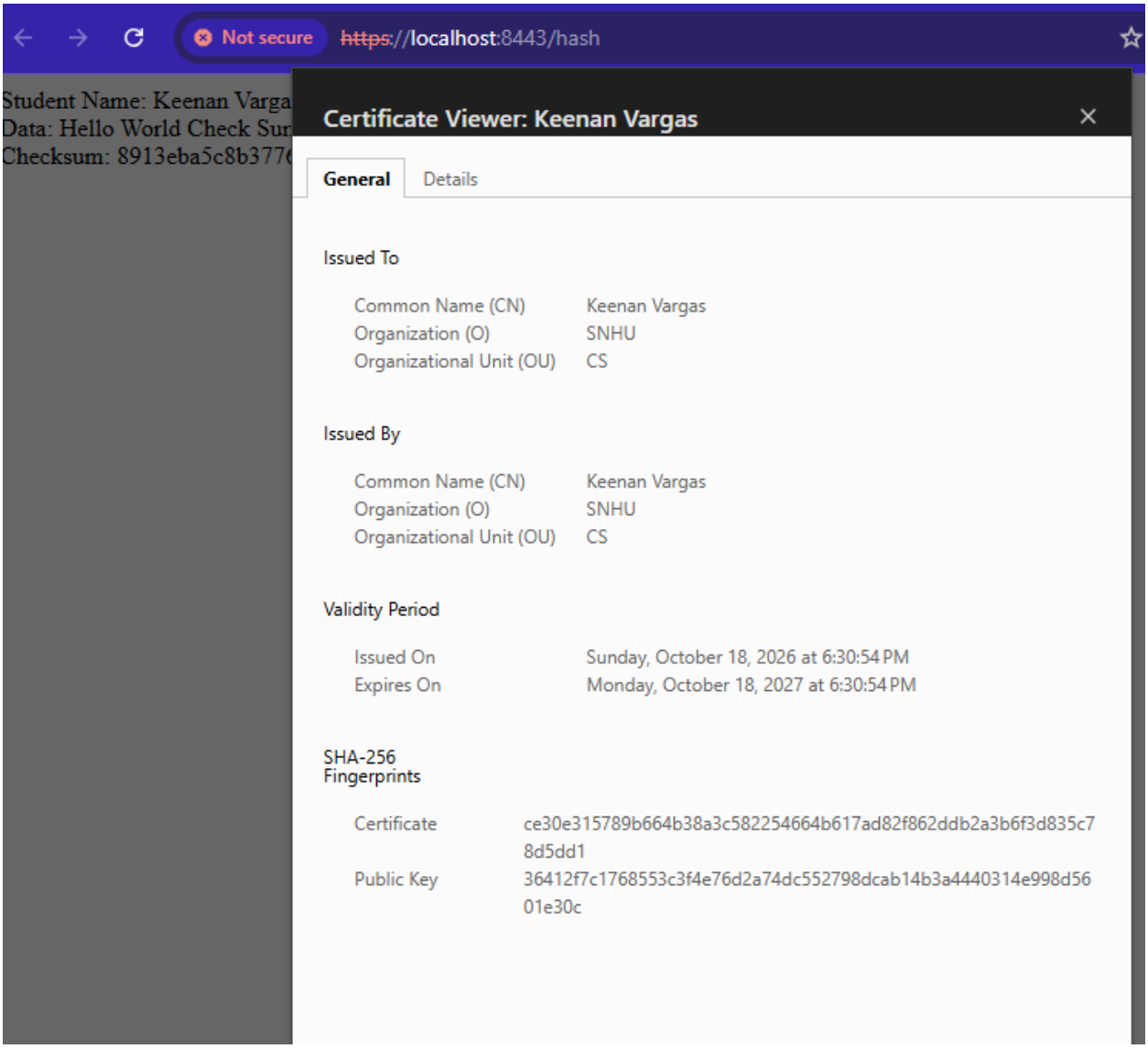
Insert a screenshot below of the checksum verification.



Student Name: Keenan Vargas
Data: Hello World Check Sum! - Keenan Vargas
Checksum: 8913eba5c8b3776e9d558d899666fbb4d97f873d862d1bb60e1c505df5aafd00

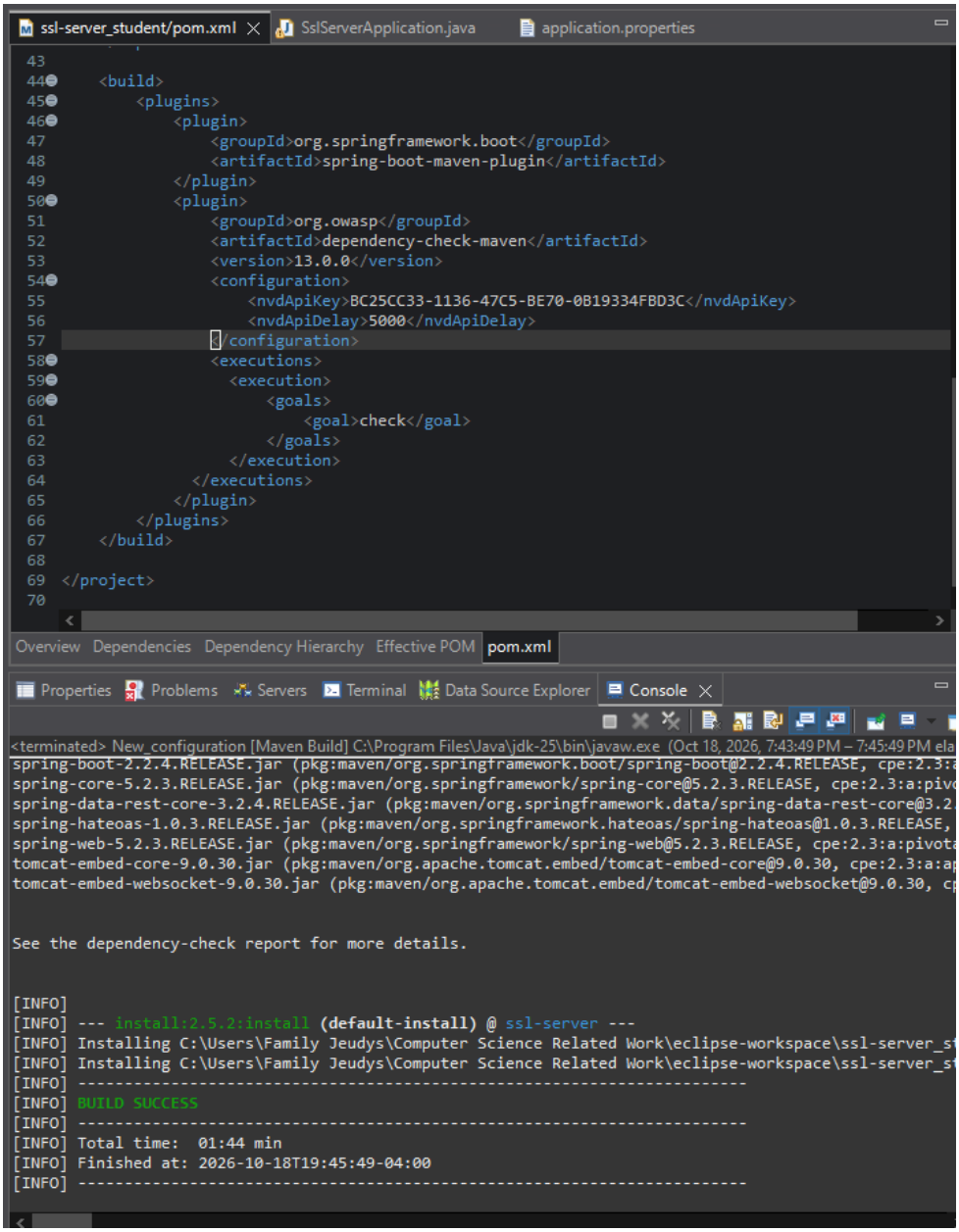
4. Secure Communications

Insert a screenshot below of the web browser that shows a secure webpage.



5. Secondary Testing

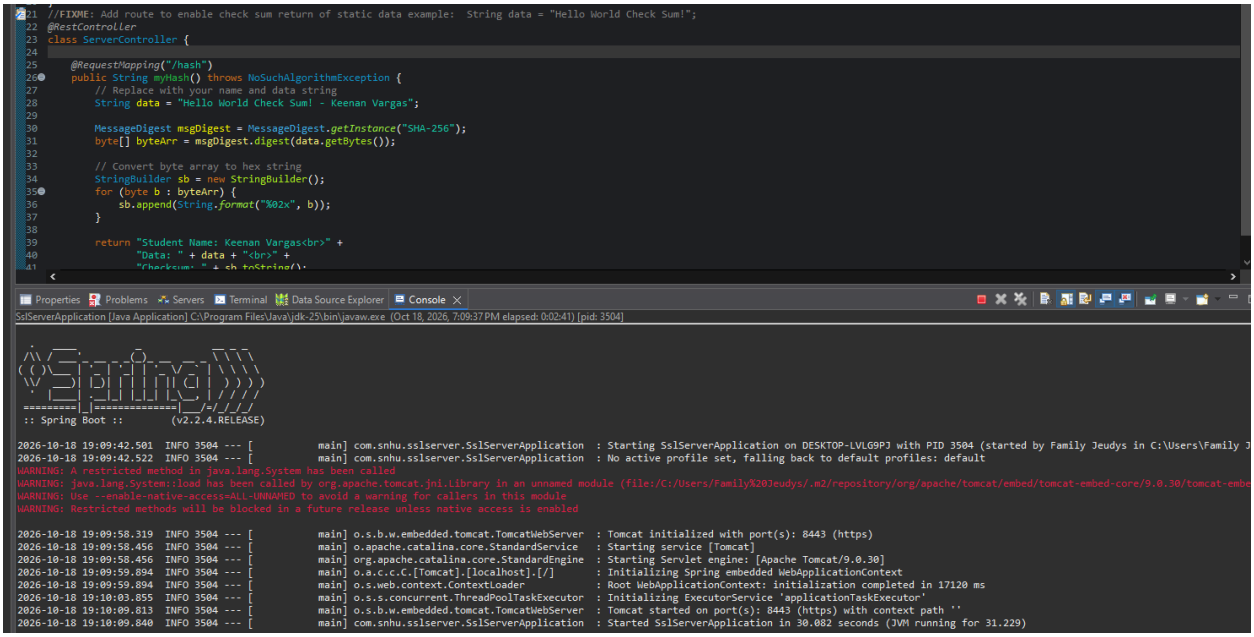
Insert screenshots below of the refactored code executed without errors and the dependency-check report.





6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.



7. Summary

Originally, the application lacked a vulnerability scan configuration in the pom.xml and had no route in the codebase to generate a data integrity checksum. In the SslServerApplication file, it contained a FIXME line where the /hash endpoint needed to be implemented to compute the SHA-256 hash function. Implementing this ensures data integrity, while configuring the application.properties file

successfully transitions the server from the unencrypted HTTP to a secure HTTPS using the generated keystore. When doing the certificate in step 2, we had to create a secure keystore password that was defined and stored in the application.properties file to allow Spring Boot to load and authenticate the sslserver.p12 keystore successfully.

8. Industry Standard Best Practices

To maintain the software application's existing security posture without breaking the core functionality and principles, industry-standard cryptographic libraries like MessageDigest for SHA-256 (in this case) and the protocols for HTTPS were used. Instead of just running custom error-prone code, we used configuration best practices like running dependency-check plugin (with an API key) and applying functional testing. These measures are crucial to create a secure coding environment for Artemis because it ensures data integrity and protection for thousands of lines with sensitive data, while also applying Artemis Financial's decision on utilizing modern web-security practices and recommended cryptographic and hashing algorithm.